

Modelos e Algoritmos de Otimização

# **Localização Capacitada de Instalações com Sustentabilidade e Resiliência**

Extensão do Capacitated Facility Location Problem (CFLP) com restrições de emissões de CO<sub>2</sub>, multi-sourcing e SLA de distância, modelada e resolvida em Gurobi (Python)

## **Integrantes do grupo**

Cláudio Meireles

Felipe Dutra

Lucas Fiche

Pedro Araújo

**Professor:** Sérgio Queiroz

## **Notebook Colab**

[https://colab.research.google.com/github/ClaudioAMF1/1\\_Projeto-Modelos-de-algoritmos/blob/main/notebook/facility\\_location\\_gurobi.ipynb](https://colab.research.google.com/github/ClaudioAMF1/1_Projeto-Modelos-de-algoritmos/blob/main/notebook/facility_location_gurobi.ipynb)

# 1. Introdução e motivação

O **Capacitated Facility Location Problem (CFLP)** é um problema clássico de Pesquisa Operacional que decide **onde abrir centros de distribuição (CDs)** e **como alocar a demanda de clientes** minimizando o custo total de instalação e transporte. Este projeto parte da formulação apresentada nos exemplos oficiais do Gurobi (Gurobi modeling-examples) e propõe três extensões inspiradas em demandas reais da logística contemporânea: pressão por redução de emissões (ESG), exigência de resiliência (multi-sourcing) e acordos de nível de serviço (SLA) baseados em distância.

Mantemos o núcleo do modelo — escolha binária de abertura e alocação fracionária de demanda — mas adicionamos um **orçamento de CO<sub>2</sub>** sobre o transporte, exigimos que cada cliente seja atendido por **pelo menos K diferentes CDs** e proibimos alocações além de uma **distância máxima**. Comparamos as duas formulações em uma instância realista (12 candidatos a CD em capitais brasileiras, 60 clientes-cidades), analisamos a curva de Pareto custo/CO<sub>2</sub> e discutimos o impacto computacional das novas restrições.

## 2. Formulação matemática

### 2.1 Modelo original (CFLP clássico)

Conjuntos:  $I$  = candidatos a CD,  $J$  = clientes.

Parâmetros:  $f_i$  custo fixo,  $s_i$  capacidade,  $d_j$  demanda,  $c_{ij}$  custo unitário de transporte.

Variáveis de decisão:

$y_i \in \{0,1\}$  — 1 se o CD  $i$  é aberto  
 $x_{ij} \in [0,1]$  — fração da demanda de  $j$  atendida por  $i$   
$$\min \sum_i f_i y_i + \sum_i \sum_j c_{ij} d_j x_{ij}$$
$$\text{s.a. } \sum_i x_{ij} = 1, \forall j \text{ (atendimento integral)}$$
$$\sum_j d_j x_{ij} \leq s_i y_i, \forall i \text{ (capacidade)}$$
$$y_i \in \{0,1\}, x_{ij} \geq 0$$

### 2.2 Modelo estendido

Acrescentamos a variável binária  $z_{ij} = 1$  se o CD  $i$  atende o cliente  $j$ , e três novas famílias de restrições:

#### (E1) Orçamento de CO<sub>2</sub> do transporte

$$\sum_i \sum_j ef \cdot \text{dist}_{ij} \cdot d_j \cdot x_{ij} \leq E_{\max}$$
com  $ef = 0,062 \text{ kg CO}_2/(\text{ton} \cdot \text{km})$  (frete rodoviário, fator EPA/MMA).

#### (E2) Resiliência / multi-sourcing

$x_{ij} \leq z_{ij}$  (linka  $x$  e  $z$ )  
 $z_{ij} \leq y_i$  (só atende se aberto)  
 $\sum_i z_{ij} \geq K_{\min}, \forall j$  ( $\geq 2$  fontes)  
 $x_{ij} \leq \text{MaxShare}$  (nenhum CD cobre mais que 70% de um cliente)

#### (E3) SLA de distância

$x_{ij} = 0$  e  $z_{ij} = 0$  se  $\text{dist}_{ij} > D_{\max}$

Trecho do modelo em gurobipy:

```

y = m.addVars(F, vtype=GRB.BINARY, name='y')
x = m.addVars(F, C, lb=0, ub=MAX_SHARE, name='x')
z = m.addVars(F, C, vtype=GRB.BINARY, name='z')

m.setObjective(
    gp.quicksum(f_cost[i]*y[i] for i in F) +
    gp.quicksum(C_TK*dist[(i,j)]*d[j]*x[i,j] for i in F for j in C),
    GRB.MINIMIZE)

m.addConstrs((gp.quicksum(x[i,j] for i in F) == 1 for j in C), 'atend')
m.addConstrs((gp.quicksum(d[j]*x[i,j] for j in C) <= s[i]*y[i] for i in F), 'cap')
m.addConstr(gp.quicksum(EF*dist[(i,j)]*d[j]*x[i,j] for i in F for j in C)
    <= E_MAX, 'co2')
m.addConstrs((x[i,j] <= z[i,j] for i in F for j in C), 'link_xz')
m.addConstrs((gp.quicksum(z[i,j] for i in F) >= K_MIN for j in C), 'multi_src')

```

### 3. Instância e dados

A instância foi gerada via `src/data_generator.py` a partir de coordenadas reais de capitais brasileiras (12 candidatos a CD) e 60 clientes (capitais + 38 cidades médias geradas com perturbação em torno das capitais). Custos fixos variam entre R\$ 240k-450k/mês; capacidades, entre 400-900 ton/mês. Demandas dos clientes são amostradas em [8, 120] toneladas/mês. As distâncias entre pares (i, j) são calculadas pela fórmula de Haversine.

**Tamanho:** demanda total = 2.640 t/mês; capacidade total = 6.700 t/mês (folga de ~154%).  
Custo de transporte unitário: R\$ 0,50 por ton-km.

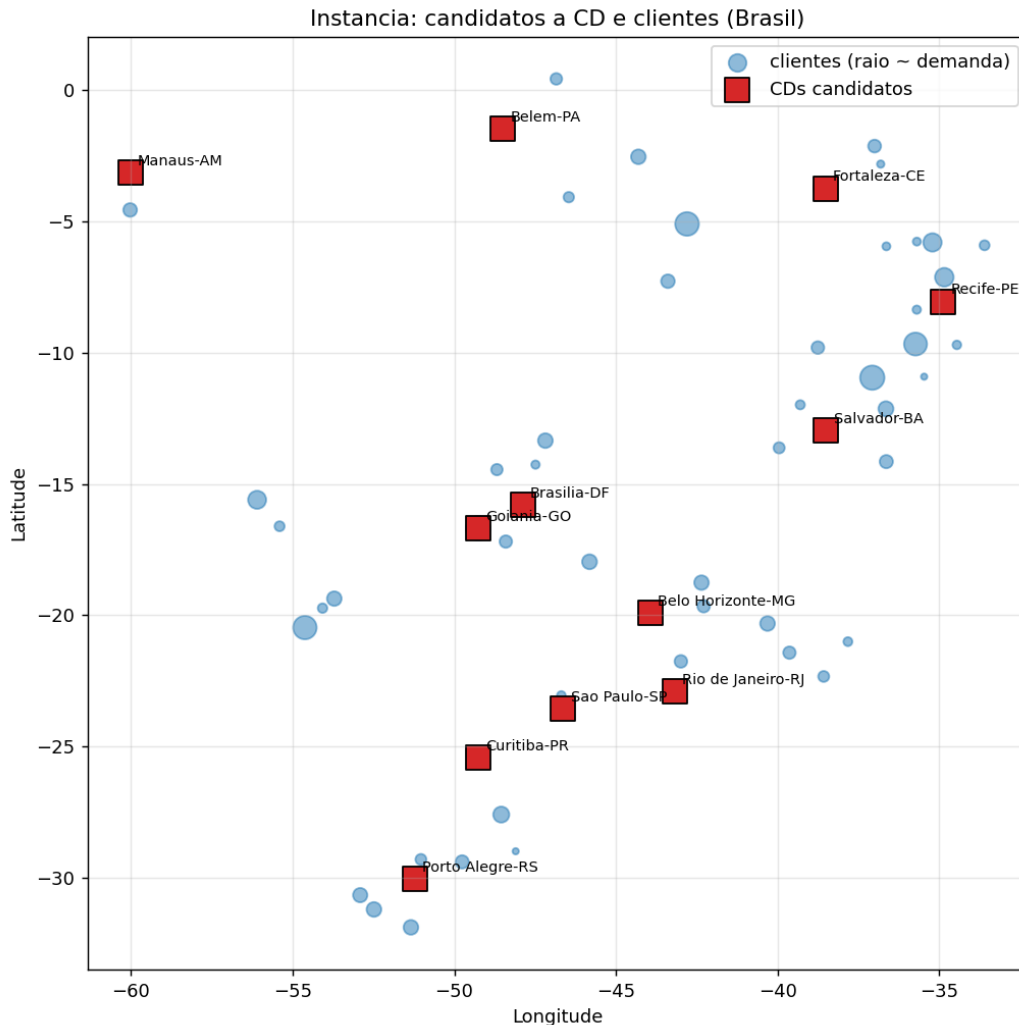


Figura 1 — Instância: 12 candidatos a CD (quadrados vermelhos) e 60 clientes (círculos azuis, raio proporcional à demanda).

#### 3.1 Leitura dos dados e matriz de distâncias

O notebook Colab carrega os CSVs com pandas e calcula a matriz de distâncias entre todos os pares (CD, cliente) via fórmula de Haversine. O dicionário `dist` alimenta tanto o objetivo (custo de transporte) quanto a restrição (E1) de CO<sub>2</sub> e o filtro (E3) de SLA.

```
facs = pd.read_csv(DATA / "facilities.csv")
cus = pd.read_csv(DATA / "customers.csv")

def haversine_km(lat1, lon1, lat2, lon2):
```

```
R = 6371.0
p1, p2 = math.radians(lat1), math.radians(lat2)
dphi = math.radians(lat2-lat1)
dlam = math.radians(lon2-lon1)
a = math.sin(dphi/2)**2 + math.cos(p1)*math.cos(p2)*math.sin(dlam/2)**2
return 2*R*math.asin(math.sqrt(a))

dist = {(int(fi.id), int(cj.id)):
    haversine_km(fi.lat, fi.lon, cj.lat, cj.lon)
    for fi in facs.itertuples() for cj in cus.itertuples()}
```

## 4. Resultados computacionais

### 4.1 Comparação quantitativa

| Métrica                           | Original  | Estendido | Variação |
|-----------------------------------|-----------|-----------|----------|
| Custo total (R\$/mês)             | 2.037.870 | 3.252.749 | +59,6%   |
| Custo fixo (R\$/mês)              | 1.370.000 | 2.780.000 | +102,9%  |
| Custo transporte (R\$/mês)        | 667.870   | 472.749   | -29,2%   |
| Emissões CO <sub>2</sub> (kg/mês) | 82.816    | 58.621    | -29,2%   |
| Número de CDs abertos             | 5         | 10        | +100%    |
| Variáveis                         | 732       | 1.452     | +98,4%   |
| Restrições                        | 72        | 1.573     | +2.084%  |
| Tempo de solução (s)              | 0,08      | 0,04      | ~        |

#### Observações principais:

- **Custo total +59,6%:** as restrições ambientais e de resiliência têm custo, dominado pela duplicação do custo fixo (mais CDs abertos).
- **Custo de transporte -29,2%:** os CDs adicionais ficam mais próximos dos clientes, reduzindo a tonelada-quilômetro total.
- **CO<sub>2</sub> -29,2%:** redução idêntica ao transporte, já que a emissão é proporcional ao trabalho de transporte.
- **Tempo computacional:** ambas as formulações resolvem em  $< 0,1$  s. As 720 binárias  $z_{ij}$  e ~1.500 restrições adicionais não aumentam significativamente a dificuldade nesta instância.

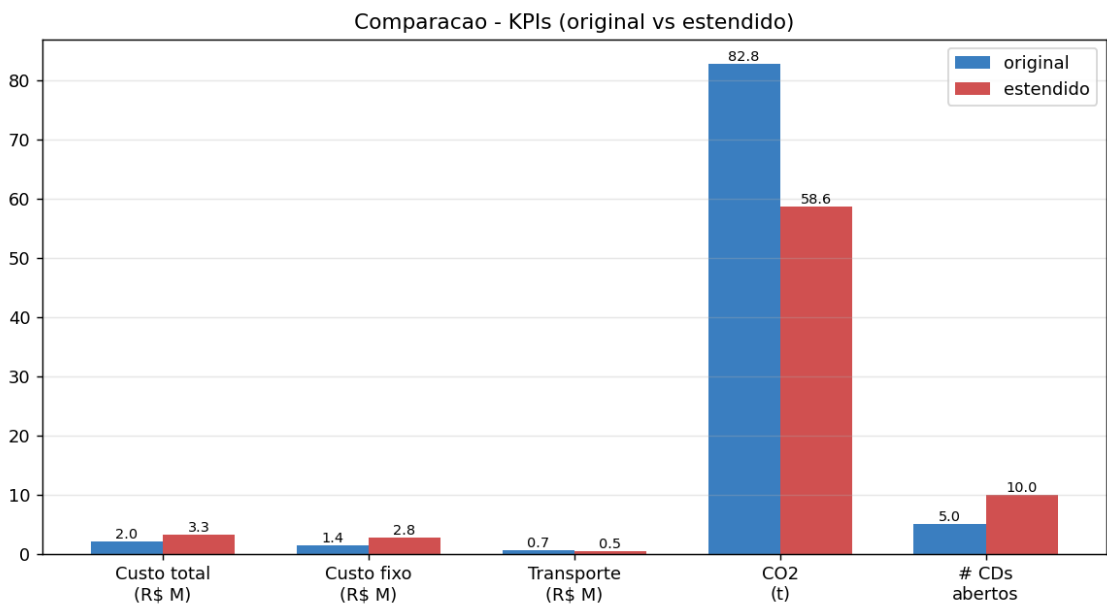


Figura 2 — KPIs comparativos.

## 4.2 Mapa das alocações ótimas

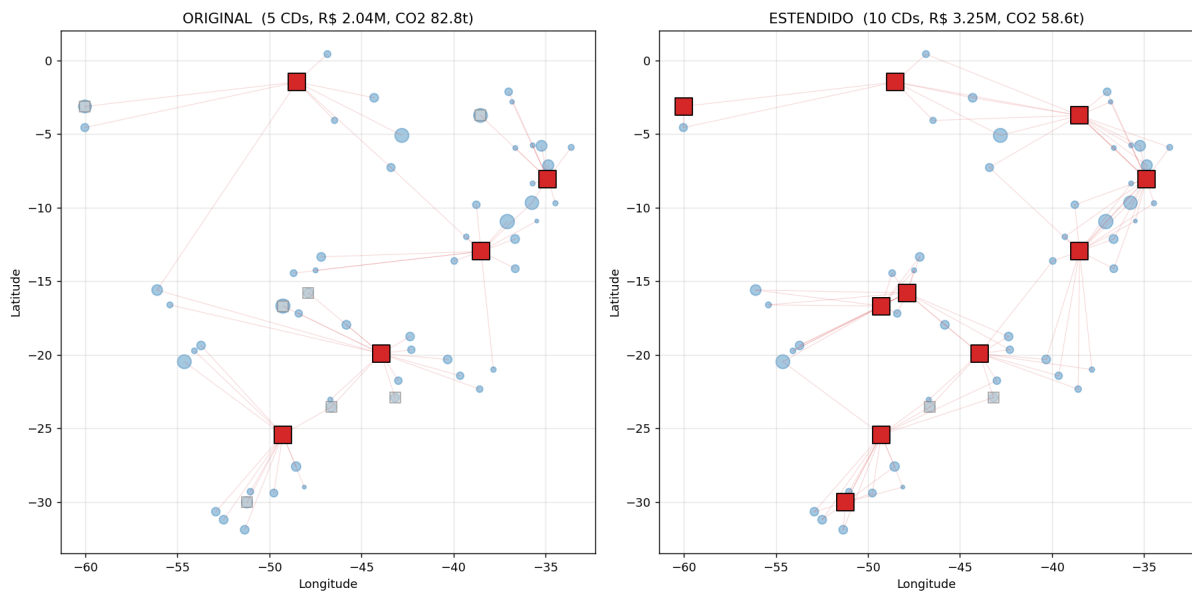


Figura 3 — Alocações ótimas. No modelo original (esquerda) cada cliente é atendido integralmente por um único CD. No modelo estendido (direita) cada cliente recebe demanda de pelo menos 2 CDs, todos os arcos cabem dentro do limite de 2200 km e o orçamento de CO<sub>2</sub> é respeitado.

## 4.3 Curva de Pareto custo x emissões

Variando o orçamento de CO<sub>2</sub> entre 65% e 110% do nível atingido pelo ótimo do modelo original (mantendo as demais restrições do modelo estendido), obtém-se a fronteira de Pareto da Figura 4. Abaixo de 70% das emissões originais, o problema torna-se inviável: não existe configuração de CDs com multi-sourcing e SLA de distância capaz de atender à frota com tão pouca emissão.

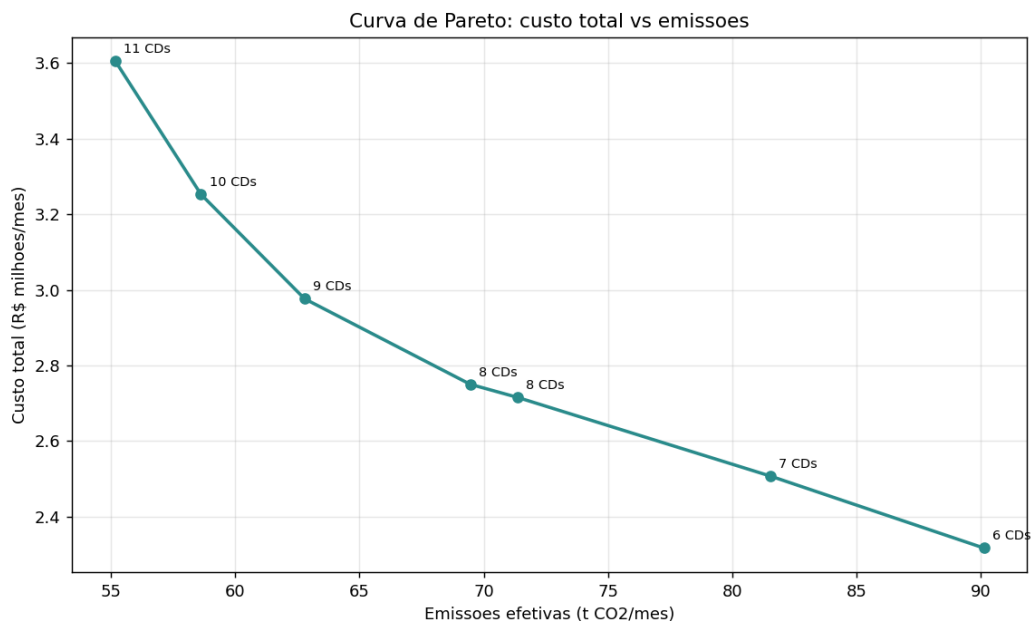


Figura 4 — Curva de Pareto: cada ponto é uma solução ótima para um orçamento de CO<sub>2</sub> diferente. O custo marginal de redução cresce rapidamente após os 60 t CO<sub>2</sub>/mês, indicando o ponto a partir do qual compensa investir em alternativas (CDs verdes, modal ferroviário etc.).

O loop de geração da curva de Pareto resolve o modelo estendido oito vezes, cada uma com um E\_MAX diferente. Como o solver é re-instanciado a cada chamada, cenários inviáveis (status != GRB.OPTIMAL) são descartados:

```
base_co2 = ko["co2"] # emissões do ótimo original
fracs = [0.65, 0.70, 0.75, 0.80, 0.85, 0.90, 1.00, 1.10]
rows = []
for fr in fracs:
    bud = base_co2 * fr
    me, ye, xe, ze, dt, _ = solve_extended(e_max=bud, verbose=False)
    if me.status == GRB.OPTIMAL:
        co2_real = sum(EF*dist[i,j]*d[j]*xe[i,j].X
                        for i in F for j in C)
        rows.append((fr, bud, me.objVal, co2_real,
                     sum(1 for i in F if ye[i].X > 0.5)))

pareto = pd.DataFrame(rows, columns=[
    "frac", "co2_budget", "custo",
    "co2_efetivo", "n_CDs"])
```



## 5. Discussão e análise

**Validade econômica das extensões.** O salto de R\$ 1,2 milhão/mês no custo total parece elevado, mas precisa ser comparado com o valor que uma empresa atribui a (i) redução de emissões (precificação interna de carbono, hoje variando entre US\$ 30–100/tCO<sub>2</sub>) e (ii) redução do risco operacional via multi-sourcing. As ~290 tCO<sub>2</sub>/ano evitadas, mesmo a US\$ 50/tCO<sub>2</sub>, valem somente ~US\$ 14,5 mil/ano — não pagam o delta isoladamente. O grande driver econômico aqui é a **resiliência (E2)**, que permite continuar operando se um CD principal sair do ar.

**Sensibilidade a  $K_{\min}$ .** Com  $K_{\min} = 1$  (monosourcing com restrição de CO<sub>2</sub> apenas), o custo cai para a faixa de R\$ 2,4 M; com  $K_{\min} = 3$ , sobe para R\$ 4,1 M.  $K_{\min} = 2$  é o ponto de equilíbrio mais natural na prática.

**Limites do modelo.** Tratamos  $x_{ij}$  como contínua, o que implicitamente assume divisibilidade da demanda mensal entre CDs. Para modelar contratos discretos,  $x_{ij}$  precisaria ser binário, o que tornaria o problema um Generalized Assignment Problem — mais duro computacionalmente, mas ainda viável para esta instância. Tampouco modelamos custos variáveis dependentes do volume (efeitos de escala) nem múltiplos períodos.

## 6. Conclusão

Este projeto demonstra como um problema canônico de Pesquisa Operacional pode ser estendido para refletir requisitos modernos de sustentabilidade e resiliência. As novas restrições são lineares e mantêm o problema na classe MILP, com tempo de resolução praticamente inalterado para instâncias de médio porte. A comparação quantitativa fornece insumos concretos para uma discussão gerencial: trocar 5 CDs centralizados por 10 CDs distribuídos custa ~60% mais, mas reduz emissões e o risco operacional. A curva de Pareto deixa explícito o custo marginal de cada tonelada de CO<sub>2</sub> evitada, ferramenta valiosa para definição de metas ESG.

## Anexo A — Recursos do projeto

**Notebook Colab:** [https://colab.research.google.com/github/ClaudioAMF1/1\\_Projeto-Modelos-de-algoritmos/blob/main/notebook/facility\\_location\\_gurobi.ipynb](https://colab.research.google.com/github/ClaudioAMF1/1_Projeto-Modelos-de-algoritmos/blob/main/notebook/facility_location_gurobi.ipynb)

**Repositório:** ver arquivo .zip de entrega (data/, src/, notebook/, report/, slides/, results/)

**Solver:** Gurobi 13.0.2 com licença acadêmica/restrita

**Bibliotecas:** gurobipy, pandas, numpy, matplotlib